

Natural Computing

LMU Munich
summer term 2025

Thomas Gabor





Werkstudent Softwareentwickler (m/w/d)

Quantum Computing, Web & KI

Deine Aufgaben:

- Software- & Schnittstellenentwicklung (REST-APIs, QC-Hardware)
- Frontend-Visualisierung mit React.js
- DevOps & Deployment (Docker, Kubernetes)
- Analyse & Entwurf einer Domänensprache (DSL)
- Hybride Algorithmen (Python, klassisch + quantenbasiert)

Dein Profil:

- Laufendes Studium IT/Physik/Mathe/Ing.
- Interesse Quantencomputing & Kommunikationstech.
- Erste Erfahrung Python/JavaScript, React.js+, REST, Docker/K8s von Vorteil

Wir bieten:

- ✓ 100% Remote
- ✓ Teilzeit
- ✓ Flexible Arbeitszeiten

Ansprechpartner:
Tobias Rohe



AIGNOSCO GmbH, München

Mind the calendar!

24	—	2025-06-12 Lecture 08
25	2025-06-17 Lecture 09	—
26	2025-06-24 Exercise 04	2025-06-26 Lecture 10

Exam Date

Tuesday, July 29, 2025

start 9:00, duration 90 minutes

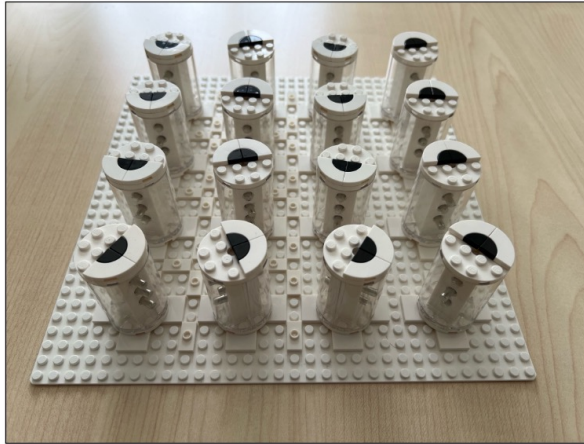
Großhaderner Str. 9, room N 00.001

details and registration soon in moodle

Previously on...



Experiment Time!



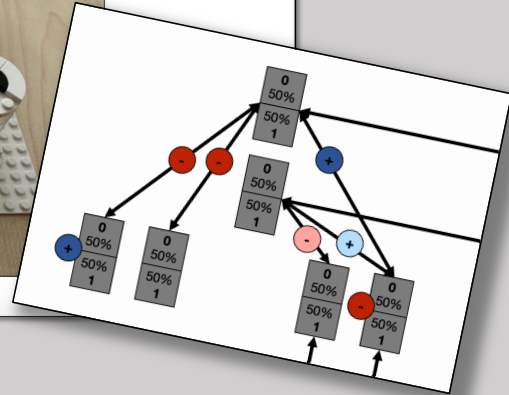
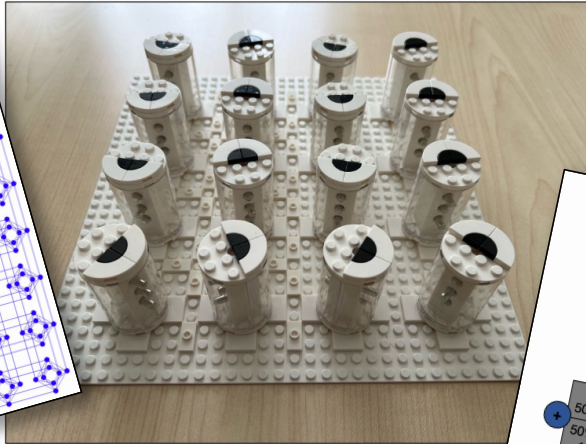
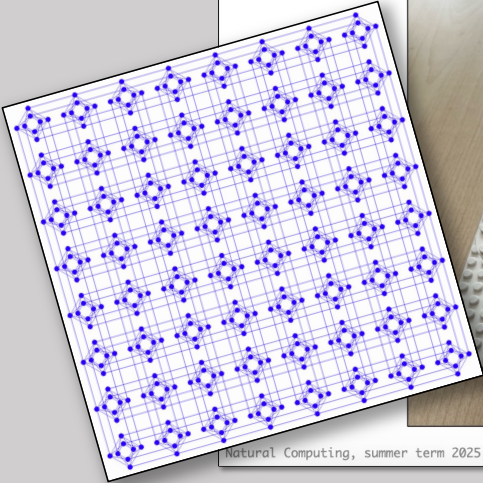
Natural Computing, summer term 2025, LMU Munich

164

Previously on...



Experiment Time!



Natural Computing, summer term 2025, LMU Munich

Definition 7 (Ising model). Let $G = (V, E)$ be a graph with vertices $V = \{v_0, \dots, v_{n-1}\}$ for $n \in \mathbb{N}$ and undirected edges $E \subseteq V \times V$. A spin configuration $x \in \mathcal{X} = (V \rightarrow \{-1, +1\})$ is a mapping of vertices to values $\{-1, +1\}$. W.l.o.g. we assume that x has the form $x(v_i) = \sigma_i$ for any $i \in \{0, \dots, n-1\}$ and a spin vector $\vec{\sigma} \in \{-1, +1\}^n$. The energy of an Ising model is then given via

$$H(\vec{\sigma}) = - \sum_{\substack{i \in \{0, \dots, n-1\} \\ j \in \{0, \dots, n-1\}}} J_{ij} \sigma_i \sigma_j - \sum_{i \in \{0, \dots, n-1\}} h_i \sigma_i$$

where $J \in \mathbb{R}^{n \times n}$ is called the interaction matrix and $\vec{h} \in \mathbb{R}^n$ is called the vector of field strengths. We also assume that for any $i, j \in \{0, \dots, n-1\}$ it holds that $i \geq j \implies J_{ij} = 0$, i.e., J is a strictly upper triangle matrix, and that $(v_i, v_j) \notin E \implies J_{ij} = 0$, i.e., J_{ij} can only have non-zero values where an edge between i and j exists in the substrate graph G .

For an Ising model given by (G, J, h) , there are processes (both artificial and natural) that produce a spin configuration $\vec{\sigma}$ so that $H(\vec{\sigma})$ is minimized.

Definition 8 (QUBO). Let $Q \in \mathbb{R}^{n \times n}$ be an arbitrary matrix called QUBO matrix. The process of finding a vector $\vec{x} \in \mathbb{B}^n$ so to

$$\text{minimize } H(\vec{x}) = x^\top Q x = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} Q_{ij} x_i x_j$$

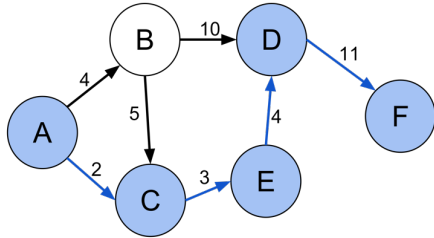
is called *quadratic unconstrained binary optimization (QUBO)*. W.l.o.g. we can assume that Q is a (non-strictly) upper triangular matrix.

Ising–QUBO Equivalence

Metaheuristic Search and Optimization

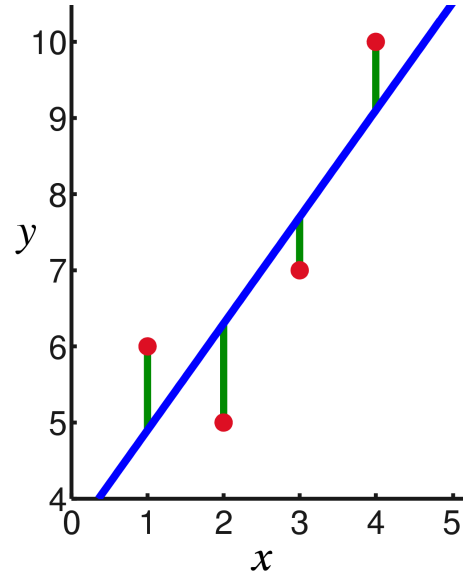
Definition 9 (optimization). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization. An optimization run of length $g + 1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

Shortest Path



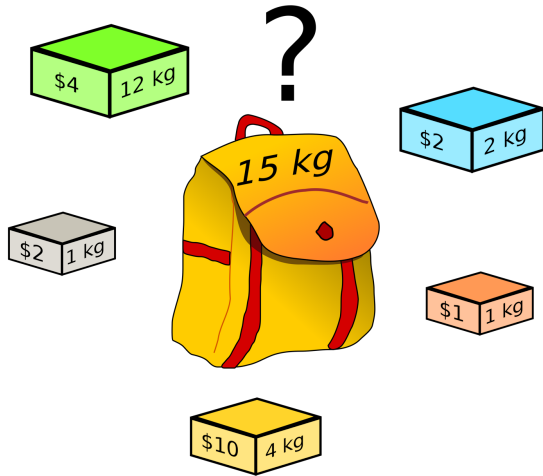
source:
[en.wikipedia.org/wiki/Shortest_path_problem#/media/
File:Shortest_path_with_direct_weights.svg](https://en.wikipedia.org/wiki/Shortest_path_problem#/media/File:Shortest_path_with_direct_weights.svg)

Linear Regression

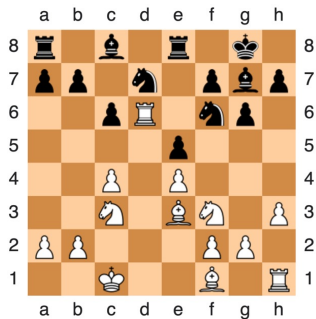


source:
[en.wikipedia.org/wiki/Linear_regression#/media/
File:Linear_least_squares_example2.png](https://en.wikipedia.org/wiki/Linear_regression#/media/File:Linear_least_squares_example2.png)

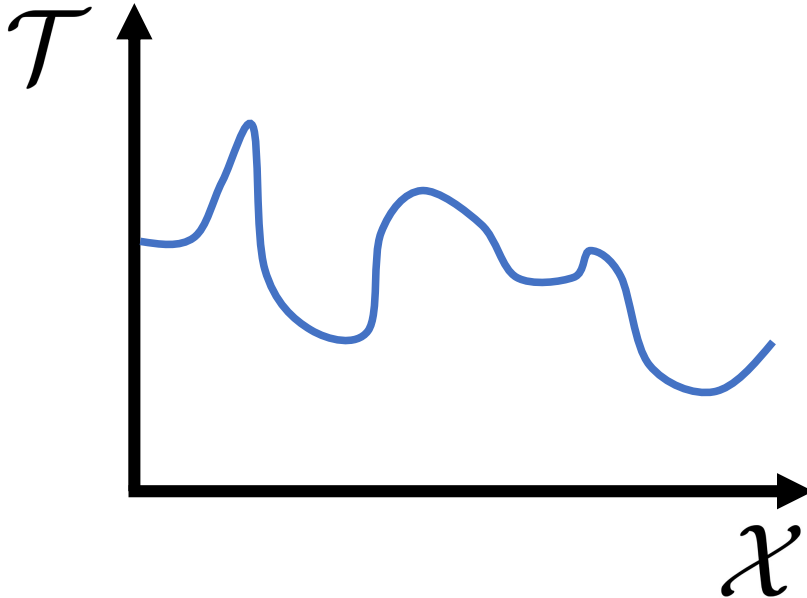
The Knapsack Problem



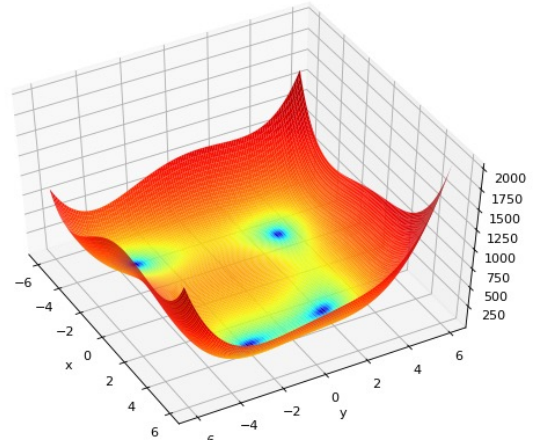
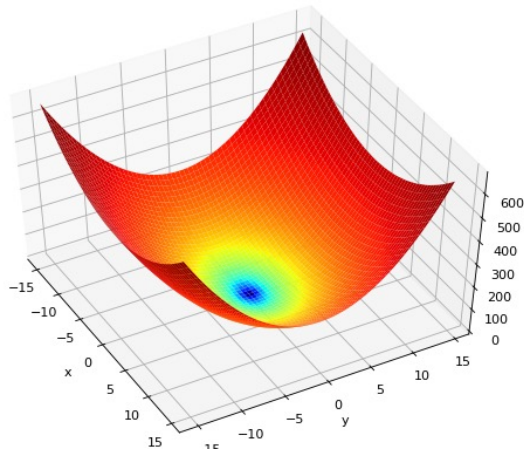
Playing Chess



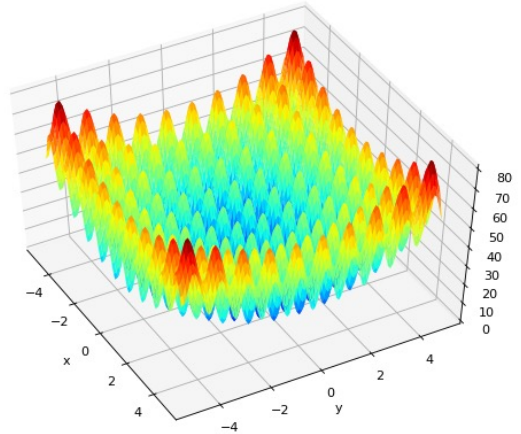
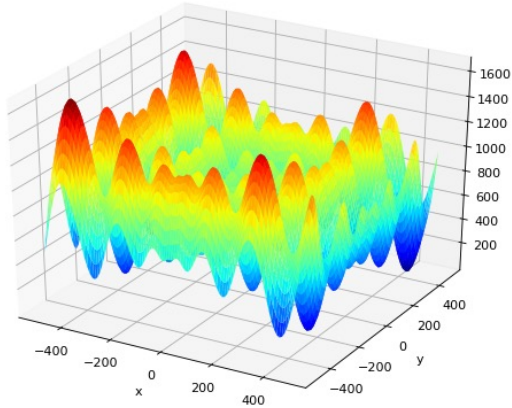
The Solution Landscape Metaphor



The Solution Landscape Metaphor



The Solution Landscape Metaphor



Definition 9 (optimization). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization. An optimization run of length $g + 1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

Definition 9 (optimization). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization. An optimization run of length $g + 1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

We use the following terminology:

- An optimization run is *working to some extent* if $t \leq t'$ for $t, t' \in [1; g] \subset \mathbb{N}$ correlates positively to some extent with $\tau(x_t) \geq \tau(x_{t'})$.
- An optimization run is *improving* if $\tau(x_t) \geq \tau(x_{t'})$ for all $t \leq t'$.
- An optimization run has *found a global optimum* iff $\tau(x) \geq \tau(x_g)$ for all $x \in \mathcal{X}$.

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t , $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{\text{def}} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

We use the following terminology:

- The *end result* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ is x_g .
- The *best result* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ is

$$*\mathcal{D} = \arg \min_{\substack{x_t \\ 0 \leq t \leq g}} \tau(x_t)$$

with the *best target value* $|\mathcal{D}| = \tau(*\mathcal{D})$.

- An optimization process $\mathcal{D}' = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x'_t \rangle_{0 \leq t \leq g'})$ is a *continuation* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ iff $g' \geq g$ and $x'_t = x_t$ for all t with $0 \leq t \leq g$. We write $\ell(\mathcal{D})$ for the set of all possible continuations of $\mathcal{D}$, which can be large if e is randomized or non-deterministic.
- An optimization process $\mathcal{D}$ is *elitist* iff for all its continuations $\mathcal{D}' = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x'_t \rangle_{0 \leq t \leq g'}) \in \ell(\mathcal{D})$ it holds that $\tau(x'_t) \geq \tau(x'_{t'})$ for all $t \leq t'$.

How do we optimize for τ ?

Algorithm 1 (single-sample random search). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. The process $\mathcal{D}$ continues via (single-sample) random search if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t, x'_t\}} \tau(x)$$

where $x'_t \sim \mathcal{X}$ is drawn at random for each call to e .

Algorithm 2 (stochastic hill climbing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. The process $\mathcal{D}$ continues via stochastic hill climbing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t, x'_t\}} \tau(x)$$

where $x'_t \sim neighbors(x_t)$ is drawn at random for each call to e .

Algorithm 3 (simulated annealing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. Let $\kappa : \mathbb{N} \rightarrow \mathbb{R}$ be a temperature schedule, i.e., a function that returns a temperature value for each time step. Let $A : \mathcal{T} \times \mathcal{T} \times \mathbb{R} \rightarrow \mathbb{P}$ with $\mathbb{P} = [0; 1] \subset \mathbb{R}$ be a function that returns an acceptance probability given two target values and a temperature. Commonly, we use

$$A(Q, Q', K) = e^{\frac{-(Q' - Q)}{K}}$$

for $\mathcal{T} \subseteq \mathbb{R}$. The process $\mathcal{D}$ continues via simulated annealing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} x'_t & \text{if } \tau(x'_t) \leq \tau(x_t) \text{ or } r \leq A(\tau(x_t), \tau(x'_t), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where $x'_t \sim neighbors(x_t)$ and $r \sim \mathbb{P}$ are drawn at random for each call to e .

Eierkartons!



source: Wikipedia

Wir brauchen Eierkartons...